

2D FFT of an Image Using CPU, Multiprocessing, and GPU

Submitted By

Name: Aryamann Srivastava

USN: 1DS24CS033

Course: Computer Science & Engineering

Date: 8/5/2026

Aim

To understand and implement 2D Fast Fourier Transform (FFT) of an image using:

1. Standard CPU implementation
2. Multiprocessing-based implementation
3. GPU-based implementation using CUDA
4. CuPy-based GPU implementation

The project also compares the execution times of the above methods for image processing.

Objective

The objective of this project is:

- To understand the working of Fast Fourier Transform (FFT)
- To implement 2D FFT using a provided radix-2 FFT algorithm
- To compare CPU and GPU performance
- To analyze the impact of multiprocessing and GPU acceleration on execution speed
- To study GPU parallel computing in image processing applications

Introduction

Fast Fourier Transform (FFT) is an efficient algorithm used to compute the Discrete Fourier Transform (DFT) of a signal or image.

In image processing, FFT converts an image from the spatial domain into the frequency domain. This helps in:

- Image filtering

- Compression
- Frequency analysis
- Signal processing
- Pattern recognition

A 2D FFT is performed by:

1. Applying 1D FFT on every row
2. Applying 1D FFT on every column

The provided radix-2 optimized FFT function was used as the base implementation for this project.

Theory

Discrete Fourier Transform (DFT)

The DFT of a sequence is defined as:

$$X(u) = \sum_{k=0}^{N-1} x(k) e^{-j2\pi uk/N}$$

where:

- $x(k)$ = input sequence
- $X(u)$ = frequency domain output
- (N) = number of samples

The direct DFT has computational complexity:

$$O(N^2)$$

which becomes very slow for large inputs.

Fast Fourier Transform (FFT)

FFT reduces computational complexity using divide-and-conquer techniques.

Complexity of FFT:

[
 $O(N \log N)$
]

The radix-2 FFT algorithm uses:

- Bit reversal ordering
- Butterfly operations
- Twiddle factors

This significantly improves performance compared to direct DFT.

Software and Libraries Used

- Python
- NumPy
- OpenCV
- Matplotlib
- Multiprocessing
- Numba CUDA
- CuPy
- Google Colab GPU Environment

Methodology

1. Sequential CPU FFT

The provided `fft_radix2_optimized()` function was used for performing 1D FFT.

2D FFT was implemented by:

1. Applying FFT on all rows
2. Applying FFT on all columns

using:

`np.apply_along_axis()`

2. Multiprocessing FFT

Python multiprocessing was used to parallelize row and column FFT operations.

Implementation used:

```
Pool(cpu_count())
```

This distributed FFT computations across multiple CPU cores.

3. GPU-based FFT using CUDA

A custom CUDA kernel was implemented using:

```
from numba import cuda
```

GPU threads processed FFT-like operations in parallel.

The GPU implementation demonstrated:

- Parallel execution
- CUDA kernel launching
- GPU memory allocation
- GPU synchronization

4. CuPy-based FFT

CuPy was used to perform FFT directly on the GPU.

Implementation used:

```
cp.fft.fft2()
```

CuPy internally uses NVIDIA cuFFT library which is highly optimized for GPU execution.

A warm-up run was performed before benchmarking to avoid CUDA initialization overhead.

Experimental Setup

- Image format: JPEG
- Image size used: 512 × 512
- GPU used: NVIDIA Tesla T4 (Google Colab)
- CUDA Version: 13.0

Results

Timing Comparison

Method	Time Taken
Sequential CPU FFT	1.5938 s
Multiprocessing FFT	8.1323 s
Custom GPU Kernel	1.917917 s
CuPy GPU FFT	0.000906 s

Observations

Sequential CPU FFT

- Successfully reconstructed image after inverse FFT
- Moderate execution speed
- Efficient compared to direct DFT

Multiprocessing FFT

- Slower than sequential execution
- Process creation and memory transfer overhead reduced efficiency
- Multiprocessing was not beneficial for 512×512 image size

Custom GPU Kernel

- Successfully executed on GPU
- Demonstrated GPU parallelism
- Slower than CuPy due to:
 - non-optimized FFT logic
 - low GPU occupancy
 - lack of radix-2 optimization

- absence of shared memory optimization

A GPU utilization warning was observed:

Grid size 8 will likely result in GPU under-utilization

This indicates that the GPU resources were not fully utilized.

CuPy GPU FFT

- Fastest implementation among all methods
- Massive acceleration achieved using GPU
- Extremely low execution time due to:
 - cuFFT optimization
 - radix-2 FFT optimization
 - CUDA acceleration
 - parallel execution on GPU cores

Performance Analysis

Why GPU Was Faster

GPU computing provides:

- Massive parallelism
- Thousands of CUDA cores
- Efficient matrix computation
- High arithmetic throughput

FFT is highly suitable for GPU acceleration because butterfly operations can execute independently in parallel.

Why Multiprocessing Was Slower

Python multiprocessing introduced:

- Process creation overhead
- Inter-process communication overhead

- Memory copying overhead
- Synchronization cost

For smaller image sizes, these overheads outweighed the benefits of parallel execution.

Why CuPy Outperformed Custom GPU Kernel

The custom GPU kernel implemented DFT-style operations whose complexity remained:

[
 $O(N^2)$
]

whereas CuPy internally uses optimized radix-2 FFT algorithms with complexity:

[
 $O(N \log N)$
]

Hence CuPy achieved significantly better performance.

Advantages of GPU Computing

- High parallel processing capability
- Faster execution for matrix and FFT operations
- Efficient for image and signal processing
- Suitable for scientific computing and AI applications

Applications of FFT

- Image compression
- Signal processing
- Audio analysis
- Medical imaging
- Communication systems
- Image filtering
- Frequency domain analysis

Conclusion

The project successfully implemented 2D FFT using multiple approaches including CPU, multiprocessing, CUDA GPU, and CuPy GPU acceleration.

The experimental results showed that:

- GPU acceleration provides enormous performance improvement for FFT operations
- Python multiprocessing is not always efficient for smaller workloads
- CuPy provides highly optimized FFT execution using NVIDIA cuFFT library
- FFT operations are highly suitable for parallel execution on GPUs

Among all implementations, the CuPy GPU FFT achieved the best performance.

References

1. NumPy Documentation
2. CuPy Documentation
3. Numba CUDA Documentation
4. OpenCV Documentation
5. NVIDIA CUDA Programming Guide
6. FFT Theory and Signal Processing Notes